# Backend Requirements

Construction UI — Extracted from Frontend

*Source: D:\construction-ui — a static HTML/vanilla-JS application with no real backend today.*

*assets/data.js currently serves as the "database" via browser localStorage.*

*This document specifies the data models, API surface, business logic, and authorization model a real backend must implement.*

## 1. Entities

Nineteen distinct data entities were identified, each currently stored as its own key in browser localStorage. Field names and types below are exactly as used in the existing code.

### User (cui_users_v3)

**id, name, username, email, phone** *string*
**roleId** *FK → Role*
**assignedProjectIds** *string[]* — M2M → Project
**monthlySalary** *number* — recurring commitment for staff, e.g. supervisors
*assignedProjectIds is kept in sync bidirectionally with each Project's user-assignment field. Actual salary disbursements are recorded separately as SalaryPayment rows.*

### Role (cui_roles_v1)

**id, name, description** *string*
**isSystem** *boolean* — seeded roles cannot be deleted
**allProjects** *boolean* — true = bypass project-assignment scoping
**permissions** *object* — per module: { create, read, update, delete }
*Permission modules: projects, labour, stock, expenses, reports, progress, customers, users, roles.*

*Seeded roles — Administrator (allProjects, full access); Site Supervisor (assigned projects only, partial access); Accountant (allProjects, full on expenses/customers, read-only elsewhere).*

### Project (cui_projects_v3)

**id, name, code, address** *string*
**clientName** *string* — denormalized display value, derived from customerIds
**customerIds** *string[]* — M2M → Customer
**startDate, targetEndDate** *ISO date*
**status** *enum* — planning | active | on_hold | completed | cancelled
**budget** *number*
**openingBalance** *number*
**openingBalanceDate** *ISO date*
**openingBalanceType** *enum* — credit | debit
**description** *string*

### Customer (cui_customers_v1)

**id, name, contactPerson, phone, email, address, gstin, notes** *string*

### CustomerLedgerEntry (cui_customer_ledger_v1)

**id, customerId, projectId** *string*
**type** *enum* — invoice | receipt
**amount, date, description, invoiceNumber** *mixed*
*Derived: balanceDue = Σ invoice − Σ receipt (per customer, or per customer + project).*

### Labourer (cui_labourers_v1 (global master))

**id, name, skill, phone** *string*
**defaultDailyWage** *number*
**isActive** *boolean*
*Skills: Mason, Helper, Electrician, Plumber, Carpenter, Painter, Welder, Bar Bender.*

### Attendance (cui_attendance_v1)

**id, labourerId, projectId, date** *string*
**daysWorked** *number* — 0.5 | 1 | 1.5 (overtime)
**wageRate** *number* — snapshotted from labourer's defaultDailyWage at entry time, editable after
**status** *enum* — present | absent
**notes, markedByUserId** *string*
*Constraint: a labourer cannot have two attendance rows on the same date across different projects — must be enforced server-side.*

*CRUD surface (from project-labour.html): POST /projects/:id/attendance · GET /projects/:id/attendance?from=&to= · PUT /attendance/:attId · DELETE /attendance/:attId*

### WagePayment (cui_wage_payments_v1)

**id, labourerId, projectId, amount** *mixed*
**type** *enum* — advance | payment
**date, notes** *string*
*Derived (getWageSummary): earned = Σ(daysWorked × wageRate); owed = earned − Σ payments.*

### SalaryPayment (cui_salary_payments_v1)

**id, userId, projectId, amount** *mixed*
**type** *enum* — advance | payment
**date, notes** *string*
*Distinct from User.monthlySalary (the ongoing commitment) and from WagePayment (labourer-specific).*

### Material (cui_materials_v1 (global master))

**id, name, category, unit** *string*
**reorderThreshold** *number*

### StockTransaction (cui_stock_txns_v1 — the most complex entity)

**id, projectId** *string* — projectId may be the sentinel "godown"
**materialId** *FK → Material*
**type** *enum* — inward | outward | adjustment
**quantity, ratePerUnit, totalAmount** *number*
**date, supplierName, invoiceNumber** *string* — supplier/invoice apply to vendor-purchase rows only
**gstPercent, gstAmount** *number*
**notes** *string*

**source** *enum* — purchase | godown | transfer | task_usage
**sourceProjectId** *string* — the "other side" of a godown/transfer move
**transferId** *string* — links the two mirrored rows of one transfer (outward + inward)
**billId** *string* — links multiple lines recorded under one godown bill
**taskId, subtaskId** *string* — present when source = task_usage
*Derived: balance(projectId, materialId) = Σ inward + Σ adjustment − Σ outward, computed live rather than stored. A low-stock alert fires when balance < material.reorderThreshold.*

*Integrity rule: transfer pairs (transferId) and bill groups (billId) must always be edited or deleted together — touching one side alone breaks stock accounting.*

### Quotation (cui_quotations_v1)

**id, materialId, storeName, price, date, notes** *mixed*
*Derived: "lowest quote per material" is computed on read.*

### Expense (cui_expenses_v1)

**id, projectId** *string*
**category** *enum* — transport | equipment_rental | permits_fees | utilities | misc
**amount, date, description** *mixed*
**taskId** *string, optional* — present only on auto-generated task_completion rows
*On read, the UI merges real Expense rows with synthesized categories: wages (from WagePayment), salary / supervisor_salary (from SalaryPayment + User.monthlySalary), and task_completion (real Expense rows auto-written when a task completes with completionCost > 0). The expenses read endpoint should return this merged view; only genuine Expense rows plus task_completion rows are directly editable or deletable.*

*The "opening balance" shown in the Expenses table equals the sum of all merged expenses dated before the selected "From" filter — a computed value, not the same as Project.openingBalance.*

### WorkReport (cui_work_reports_v1)

**id, projectId, date, title, description** *mixed*
**source** *enum* — manual | task_complete
**taskId** *string* — set when source = task_complete

### ProgressUpdate (cui_progress_v1)

**id, projectId, date** *mixed*
**percentComplete** *number*
**notes** *string*
*Derived (estimateCompletion): a linear projection of finish date from the latest percentage versus elapsed time between startDate and targetEndDate; also flags an at-risk project.*

### Task (cui_tasks_v3)

**id, projectId, title** *string*
**assignee** *string* — free text — not a User foreign key
**status** *enum* — todo | in_progress | done | <custom TaskStatusDef.key>

**`date`** *ISO date* — estimated completion
**`startDate`** *ISO date*
**`notes`** *string*
**`customFields`** *object* — schema defined by TaskFieldDef
**`subtasks`** *array* — { id, title, done, area?, ratePerUnit?, cost? }
**`materialsUsed`** *array* — { id (matches a StockTransaction id), materialId, qty, subtaskId? }
**`createdAt, createdBy, completedAt`** *mixed* — createdBy is a userId
**`sqft, completionCost, personsCount, personInCharge, durationDays`** *mixed*
**`materialUsed`** *string* — legacy free-text summary field
*getTaskCompletionSummary prefers subtask roll-ups (area, cost, rate, duration) over task-level fields whenever subtasks exist — this precedence must be replicated server-side.*

*Recording materials used on a task or subtask writes a real outward StockTransaction (source: task_usage); removing or editing that usage must restore or adjust stock in both directions.*

### TaskActivity (cui_task_activity_v1 — append-only audit log)

**`id, taskId, date, message`** *mixed*

### TaskFieldDef (cui_task_field_defs_v1 — per-project custom task field schema)

**`id, projectId, key, label`** *mixed*
**`type`** *enum* — text | number | date
*No management UI currently exists for this entity — likely a planned feature; the data model is already in place.*

### TaskStatusDef (cui_task_status_defs_v1 — per-project extra Kanban columns)

**`id, projectId, key, label`** *mixed*
**`order`** *number*

## 2. Implied REST API Surface

One resource block per entity, inferred from the CRUD operations each page performs.

| Resource | Endpoints |
|---|---|
| Auth | POST /auth/login · POST /auth/logout · POST /auth/reset-password · POST /auth/invite |
| Users | Full CRUD at /users |
| Roles | Full CRUD at /roles (block delete if isSystem or role is in use) |
| Projects | Full CRUD at /projects (plus side effects: keep assignedProjectIds and customerIds in sync) |
| Customers | Full CRUD at /customers |
| Customer Ledger | Full CRUD at /customers/:id/ledger |
| Labourers | Full CRUD at /labourers |
| Attendance | CRUD at /projects/:id/attendance (see uniqueness constraint above) |
| Wage Payments | CRUD at /projects/:id/wage-payments |
| Salary Payments | CRUD at /projects/:id/salary-payments |
| Materials | Full CRUD at /materials |
| Stock Transactions | CRUD at /projects/:id/stock and /godown/stock (preserve transferId / billId pairing) |
| Quotations | Full CRUD at /quotations |
| Expenses | CRUD at /projects/:id/expenses (GET returns the merged view described in Section 1) |
| Work Reports | Read/filter at /projects/:id/reports; create manual entries |
| Progress Updates | Create and list at /projects/:id/progress |
| Tasks | Full CRUD at /projects/:id/tasks, plus subtask CRUD, materials-used CRUD, and activity-log append |
| Task Field / Status Defs | CRUD at /projects/:id/task-fields and /projects/:id/task-statuses |

## 3. Business & Derived Logic

These values are computed on demand in the current app rather than stored as columns. They should be implemented as query endpoints or database views, not persisted fields — callers always expect them freshly calculated.

- getWageSummary(labourerId, projectId) → earned, paid, owed
- getStockBalance / getAllStockBalances → running balance per material and project
- getLowStockAlerts → balance below reorderThreshold
- getProjectCost → stockCost + wagesPaid + supervisorSalary + otherExpenses (cash-basis; excludes accrued but unpaid wages)
- estimateCompletion → linear ETA projection plus an at-risk flag

- getWagesDueTotal, getSupervisorSalaryTotal
- getCustomerLedger / getCustomerTotalExpenses → balance-due rollups
- getMaterialCostBreakdown
- getProjectStatusColumns → merges default and custom Kanban columns
- getTaskProgress / getTaskCompletionSummary → applies the subtask roll-up precedence described in Section 1

## 4. Authentication & Authorization

The current implementation is a stub: there is no password check, sessions are held only in browser sessionStorage, and the invite/reset flows merely show a confirmation toast without doing anything server-side.

However, the authorization model itself is fully designed in the frontend and used throughout the UI — it is worth preserving exactly:

- Role.permissions[module][action] — per-module create/read/update/delete gating
- Role.allProjects — a global override that bypasses project-assignment scoping
- User.assignedProjectIds — per-user project scoping when allProjects is false
- Frontend call sites to mirror one-to-one: App.can(module, action), App.guardProjectAccess(projectId), App.visibleProjects()

**What needs to be built for real:**

- Password hashing and a genuine login endpoint issuing a session or JWT
- Password-reset flow backed by real tokens — the UI already has the newPassword / confirmPassword fields waiting for a real endpoint
- Invite flow with real tokens and email delivery

## 5. Pages → Feature / CRUD Map

All 23 pages in the application and the backend feature each one implies.

| Page | Feature |
|---|---|
| index.html | Login (currently a stub) |
| reset-password.html | Password reset (currently a stub) |
| dashboard.html | Cross-project read-only aggregates |
| projects.html | Project CRUD |
| project-overview.html | Project read-only summary |
| project-tasks.html / project-task-detail.html | Task, subtask, and activity CRUD; custom Kanban columns |
| project-labour.html | Attendance CRUD |
| project-wages.html | Wage payment CRUD plus wage summary |
| project-salary.html | Salary payment CRUD plus inline monthlySalary edit |
| project-stock.html | Stock transaction CRUD (inward / outward / adjustment / transfer) |
| project-expenses.html | Expense CRUD plus the merged expenses view |
| project-progress.html | Progress update create and list |
| project-reports.html | Work report read and filter |
| godown.html | Stock transactions at the godown scope; multi-line bills |
| godown-materials.html | Material CRUD; purchase history |
| quotation.html | Quotation CRUD |
| customers.html | Customer CRUD |
| customer-detail.html | Customer ledger entry CRUD |
| labour-list.html / labourer-detail.html | Labourer CRUD; attendance and payment history |
| user-list.html | User CRUD; invite and reset stubs |
| roles.html | Role CRUD |
| profile.html | Current-user view; change password (stub) |

## 6. Integrity Rules to Encode as Database Constraints or Transactions

- Attendance: a unique constraint on (labourerId, date) — globally unique per labourer per date, not merely per project.
- Stock transaction transfer pairs (transferId): always insert, update, or delete both sides within a single transaction.
- Stock transaction bill groups (billId): multi-line inserts and deletes should be atomic.

- Task completion cascade: completing a task or subtask should atomically write a WorkReport, write an Expense (task_completion, when completionCost > 0), and keep any already-recorded materials usage consistent.
- Role deletion: block if the role is a system role or if any User still references it.
- Project deletion: decide and enforce a cascade or restrict policy for Tasks, Expenses, Stock Transactions, and Attendance tied to that project — this is not specified by the current frontend and needs an explicit decision.